

Relatório Técnico

Processo de Testes na API Itaim Vende

Paulistana - 2026

Curso: Tecnologia em Análise e Desenvolvimento de Sistemas - 4° Módulo

Disciplina: Engenharia de Software III

Docente: Prof. Me. Julian Rodrigues Valério

Equipe:

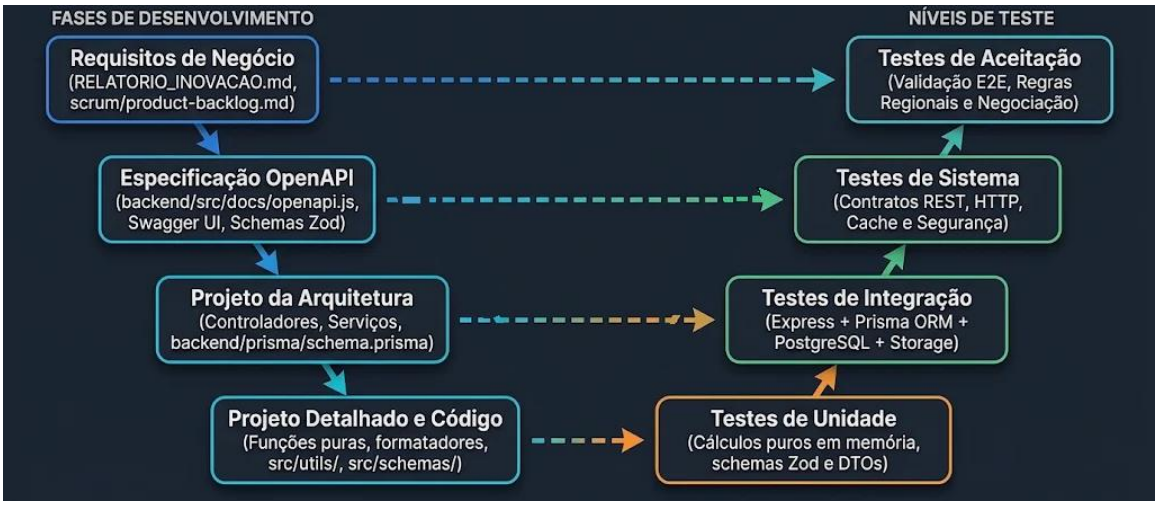
- Kayky Terles Ferreira Feitosa (Product Owner / Backend Developer)
- Pedro Ravy Teixeira de Sousa (Backend Developer)
- Josimar Rodrigues de Macedo Filho (Scrum Master Sprint 1 / Frontend Developer)
- Giselly Carvalho Cavalcante (Scrum Master Sprint 2 / Frontend Developer)
- Raíli Saweni de Sousa Reis (Frontend Developer)

Objeto sob Teste: Itaim Vende API RESTful (/api/v1) — Node.js 20.x, Express, Prisma ORM e PostgreSQL 15

Data: 08/11/2026 | **Versão da Suíte:** 2.0.0 (88 testes automatizados, 97,03% de cobertura de regras de negócio)

PARTE 1 — Testes no Ciclo de Vida: Modelo V

O Modelo V estabelece que para cada estágio do ciclo de desenvolvimento tenha um nível correspondente de teste, planejado desde o início dos requisitos. Na API Itaim Vende, essa correlação garante rastreabilidade bidirecional estrita entre a especificação de engenharia e os procedimentos de garantia da qualidade.



1. Mapeamento das Fases do Modelo V aos Artefatos Reais do Repositorio

Fase de	Nível de	Artefatos Reais do Repositório	Escopo e

Desenvolvimento	**Teste Correspondente**		**Aplicação Técnica na API Itaim Vende**
1. Requisitos de Negócio	Teste de Aceitação	`RELATORIO_INOVACAO.md`, `scrum/product-backlog.md`	Avaliação ponta a ponta dos fluxos centrais: negociação regional via WebSockets com contrapropostas e checkout com travamento compulsório de integridade financeira no servidor.
2. Especificação do Sistema	Teste de Sistema	`backend/src/docs/openapi.js`, `swagger-ui`, `backend/src/schemas/`	Verificação do comportamento de caixa-preta da API: aderência rigorosa ao contrato OpenAPI 3.0.3, diretivas de cabeçalho `Cache-Control` em categorias e busca com normalização fonética `unaccent`.
3. Projeto da Arquitetura (Camadas, ORM, Banco e Nuvem)	Teste de Integração	`backend/prisma/schema.prisma`, `backend/src/controllers/`, `backend/src/services/`	Validação da cooperação entre controladores Express, middlewares RBAC, transações atômicas via Prisma ORM

			no PostgreSQL 15 e purga física de arquivos no Supabase Storage.
4. Projeto Detalhado e Código (Algoritmos e Estruturas de Dados)	Teste de Unidade	`backend/src/utils/sanitize.js`, `backend/src/services/account/cooldown.service.js`, `backend/src/schemas/`	Execução isolada em memória (< 1s) para cálculo puro de carência temporal de datas, sanitização recursiva contra Cross-Site Scripting (XSS) e validação de limites de strings e tipos monetários.

2. Especificação de 8 Casos de Teste Concretos

Nível 1: Teste de Aceitação (Fase: Requisitos de Negócio)

Caso de Teste CTA-01

- **Identificador:** CTA-01
- **Nível:** Teste de Aceitação
- **Fase Correspondente:** Requisitos de Negócio (RELATORIO_INOVACAO.md — Eixo Tempo Real e Eventos; scrum/product-backlog.md — H7)

Entrada (Payload / Parâmetros / Headers):

- Autenticação: Token Bearer JWT de comprador ativo (role: BUYER).
- Requisição: `POST/api/v1/chat/conversations/conv_paulistana_01/messages`
- Headers: `Authorization: Bearer <jwt_comprador>`, `Content-Type: application/json`
- **Payload JSON:**

```
{
```

```
    "content": "Aceita fechar o lote de mel silvestre por R$ 180,00 para retirada no centro?",
    "offerPrice": 180.00
  }
```

Resultado Esperado:

- Status HTTP 201 Created.
- Corpo da resposta contendo o identificador gerado da mensagem, status da proposta "PENDING", timestamp oficial e offerPrice: 180.00.
- Despacho em tempo real do evento Socket.IO chat:message para a sala privada do vendedor.
- Caso o vendedor esteja desconectado (isUserOnline === false), enfileiramento assíncrono de notificação por e-mail transacional.

Caso de Teste CTA-02

- **Identificador:** CTA-02
- **Nível:** Teste de Aceitação
- **Fase Correspondente:** Requisitos de Negócio (`RELATORIO_INOVACAO.md` — Eixo Regra de Negócio; `scrum/product-backlog.md` — H8)

Entrada (Payload / Parâmetros / Headers):

- Autenticação: Token Bearer JWT de comprador autenticado.
- Requisição: `POST /api/v1/orders`
- Headers: `Authorization: Bearer <jwt_comprador>`, `Content-Type: application/json`
- Payload JSON com tentativa maliciosa de adulteração de preço no cliente:

```
{
  "items": [
    {
      "productId": "prod_artesanato_couro_01",
      "quantity": 2,
      "unitPrice": 1.50
    }
  ]
}
```

Observação: No banco de dados PostgreSQL, o produto `prod_artesanato_couro_01` possui preço de tabela fixado em `R$ 120.00`.

Resultado Esperado:

- Status HTTP `201 Created`.
- O backend descarta compulsoriamente o campo `unitPrice: 1.50` enviado pelo cliente.
- O serviço busca o valor nominal real no catálogo através do Prisma ORM (`R$ 120.00`), calculando o montante total exato de `R$ 240.00` (`totalAmount: 240.00`).
- Registro do pedido gravado no PostgreSQL com o valor íntegro, bloqueando fraude financeira.

Nível 2: Teste de Sistema (Fase: Especificação do Sistema / OpenAPI)

Caso de Teste CTS-01

- **Identificador:** CTS-01
- **Nível:** Teste de Sistema
- **Fase Correspondente:** Especificação OpenAPI (`backend/src/docs/openapi.js` — `/api/v1/products`; `scrum/product-backlog.md` — H5)

Entrada (Payload / Parâmetros / Headers):

- Requisição: `GET /api/v1/products?search=eletronicos&city=Paulistana&page=1&limit=10&sort=menor_preco`
- Headers: `Accept: application/json` (Acesso público sem token).
- Contexto da base: Anúncio cadastrado com título `"Aparelho de Som e Eletrônicos em Paulistana"`.

Resultado Esperado:

- Status HTTP `200 OK`.
- Corpo da resposta compatível com o schema OpenAPI `ProductListResponse`:

```
{
  "success": true,
  "data": [ ... ],
  "pagination": {
    "page": 1,
    "limit": 10,
```

```
    "total": 1,
    "totalPages": 1
  }
}
```

- O produto é retornado com sucesso graças à normalização fonética da extensão PostgreSQL `unaccent` combinada com o filtro geográfico estrito `city = "Paulistana"`.

Caso de Teste CTS-02

- **Identificador:** CTS-02
- **Nível:** Teste de Sistema
- **Fase Correspondente:** Especificação OpenAPI (`backend/src/docs/openapi.js` — `/api/v1/categories`; Requisito REQ-2)

Entrada (Payload / Parâmetros / Headers):

- Requisição: `GET /api/v1/categories`
- Headers: `Accept: application/json`

Resultado Esperado:

- Status HTTP `200 OK`.
- Presença mandatória do cabeçalho de resposta: `Cache-Control: public, max-age=300`
- Corpo da resposta contendo array de categorias ativas acompanhadas do somatório agregado de anúncios ativos (`activeProductsCount`).

Nível 3: Teste de Integração (Fase: Projeto da Arquitetura)

Caso de Teste CTI-01

- **Identificador:** CTI-01
- **Nível:** Teste de Integração
- **Fase Correspondente:** Projeto da Arquitetura (`backend/prisma/schema.prisma` — campo `usernameLastChangedAt`; `scrum/product-backlog.md` — H13)

Entrada (Payload / Parâmetros / Headers):

- Autenticação: Token Bearer JWT de usuário cujo `usernameLastChangedAt` está gravado no PostgreSQL há 5 dias.
- Requisição: `PATCH /api/v1/account/username`

- Headers: `Authorization: Bearer <jwt_usuario>`, `Content-Type: application/json`
- Payload JSON:

```
{
  "newUsername": "novo_vendedor_itaim"
}
```

Resultado Esperado: Status HTTP `403 Forbidden`.

Resposta padronizada de erro:

```
{
  "success": false,
  "error": {
    "code": "USERNAME_COOLDOWN_ACTIVE",
    "message": "Você só pode alterar seu nome de usuário uma vez a cada 30 dias.",
    "details": {
      "daysRemaining": 25
    }
  }
}
```

- Nenhuma alteração persistida na tabela `User` do PostgreSQL 15 via Prisma.

Caso de Teste CTI-02

- **Identificador:** CTI-02
- **Nível:** Teste de Integração
- **Fase Correspondente:** Projeto da Arquitetura (`backend/src/services/moderation/report.service.js`, Supabase Storage Client; `scrum/product-backlog.md` — H6 e H9)

Entrada (Payload / Parâmetros / Headers):

- Autenticação: Token Bearer JWT com papel de moderador (`role: MODERATOR`).
- Requisição: `DELETE /api/v1/reports/rep_infracao_89/product`
- Headers: `Authorization: Bearer <jwt_moderador>`
- Contexto da base: Anúncio associado à denúncia possui a imagem `anuncio_89_capa.webp` hospedada no bucket `products` do Supabase Storage.

Resultado Esperado:

- Status HTTP `204 No Content`.
- O produto é excluído da tabela `Product` com encerramento atômico da denúncia.
- O adaptador do Supabase Storage executa o comando `storage.from('products').remove(['anuncio_89_capa.webp'])`, garantindo a purga física do binário na nuvem sem deixar arquivos órfãos.

Nível 4: Teste de Unidade (Fase: Projeto Detalhado e Código)

Caso de Teste CTU-01

- **Identificador:** CTU-01
- **Nível:** Teste de Unidade
- **Fase Correspondente:** Projeto Detalhado e Código (`backend/src/services/account/cooldown.service.js`)

Entrada (Parâmetros de Função Pura em Memória):

- Invocação da função pura `calculateCooldownRemainingDays(lastChangedDate, cooldownPeriodDays)`:
- Parâmetro 1 (`lastChangedDate`): Timestamp equivalente a exatamente 10 dias atrás (`new Date(Date.now() - 10 * 24 * 60 * 60 * 1000)`).
- Parâmetro 2 (`cooldownPeriodDays`): Constante inteira `30`.

Resultado Esperado:

- Retorno exato de objeto contendo:
- `isBlocked`: `true` (booleano estrito).
- `daysRemaining`: `20` (número inteiro estrito).
- Execução imediata em memória (< 1 ms), sem conexões de rede ou banco de dados.

Caso de Teste CTU-02

- **Identificador:** CTU-02
- **Nível:** Teste de Unidade
- **Fase Correspondente:** Projeto Detalhado e Código (`backend/src/utils/sanitize.js`)

Entrada (Parâmetros de Função Pura em Memória):

- Invocação da função `sanitizeInput(rawPayload)` passando objeto arbitrário contendo vetores de injeção XSS aninhados:

```
{
  title: "Bode Castrado <script>alert('xss')</script>",
  description: "<img src=x onerror=alert(document.cookie)> Excelente animal.",
  details: {
    location: "Paulistana <svg onload=fetch('http://attacker.com')>"
  }
}
```

Resultado Esperado:

- Retorno estruturado do objeto sanitizado com neutralização de todas as tags ativas de script:
- `title`: `"Bode Castrado "`
- `description`: `" Excelente animal."`
- `details.location`: `"Paulistana "`
- Preservação da integridade tipográfica das strings seguras.

3. Classificação em Verificação vs. Validação dos 8 Casos de Teste

ID do Caso	Nível de Teste	Classificação	Justificativa Sintética Baseada nos Princípios de Boehm
CTA-01	Aceitação	Validação	Responde "Estamos construindo o produto certo?" ao validar se a negociação em tempo real atende às necessidades negociais do comprador e vendedor regional.
CTA-02	Aceitação	Validação	Responde "Estamos construindo o produto certo?" ao certificar que a plataforma protege o patrimônio dos usuários travando a integridade financeira no checkout.
CTS-01	Sistema	Verificação	Responde "Estamos construindo certo o produto?" ao verificar a conformidade técnica da busca fonética com os parâmetros de consulta e paginação do OpenAPI.
CTS-02	Sistema	Verificação	Responde "Estamos construindo certo o produto?" ao verificar a presença correta da diretiva e do valor do cabeçalho HTTP `Cache-Control` exigido pelo contrato.
CTI-01	Integração	Verificação	Responde "Estamos construindo certo o produto?" ao verificar se as camadas de Controller, Serviço e Prisma interagem

			corretamente ao impor a trava de data no banco.
CTI-02	Integração	Verificação	Responde "Estamos construindo certo o produto?" ao verificar a comunicação orquestrada entre o banco PostgreSQL e o bucket do Supabase Storage na exclusão física.
CTU-01	Unidade	Verificação	Responde "Estamos construindo certo o produto?" ao verificar a precisão aritmética da função matemática pura de cálculo de carência de dias em memória.
CTU-02	Unidade	Verificação	Responde "Estamos construindo certo o produto?" ao verificar se o algoritmo de sanitização de strings remove sistematicamente padrões de tags maliciosas.

4. Reflexão Crítica do Modelo V Aplicada à API Itaim Vende

Os testes unitários são ótimos, mas têm um grande ponto cego: eles não conseguem prever o caos de várias coisas acontecendo ao mesmo tempo. Na nossa API do Itaim Vende, o teste unitário do carrinho garante que a matemática da compra está certa. Mas e se dois clientes tentarem comprar a última unidade de um produto na mesma fração de segundo? O teste unitário não enxerga essa 'disputa'. Apenas os testes de integração, conectados ao nosso banco de dados real, conseguem provar que o sistema é ágil o suficiente para vender o produto para o primeiro cliente e barrar a segunda compra com segurança, sem problemas.

PARTE 2 — Testes nos Modelos Ágeis (Scrum & XP)

Nos modelos ágeis, o processo de garantia da qualidade passa a ser uma atividade contínua de engenharia, conduzida iterativamente em conjunto com o desenvolvimento de cada história de usuário ao longo das sprints de uma semana.

1. Quatro Histórias de Usuário no Formato BDD (Dado / Quando / Então)

História 1 (H5): Busca Fonética Normalizada Sem Acentos (Inovação Regional)

Descrição Formal:

- **Como** comprador do semiárido piauiense que digita sem acentuação em teclados móveis,
- **quero** buscar produtos por palavras-chave ignorando acentos, cedilhas e caixas altas/baixas,
- **para que** eu encontre produtos regionais rapidamente sem que erros de digitação impeçam a compra.

Cenário 1 — Localização de produtos com acentos a partir de busca simplificada

```
  Dado que existem produtos cadastrados no catálogo como "Eletrônicos" e "Chapéu de
Couro"

  Quando o consumidor envia uma requisição GET /api/v1/products?search=chapeu

  Então o sistema retorna status HTTP 200 OK

  E o corpo da resposta lista o produto "Chapéu de Couro" com metadados de paginação
estruturados.



Cenário 2 — Rejeição de consulta com ordenação fora do vocabulário permitido

  Dado que o consumidor submete uma busca informando o parâmetro inválido
"?sort=antigos"

  Quando o schema Zod da camada de entrada intercepta a requisição

  Então o sistema retorna status HTTP 400 Bad Request

  E a resposta informa que o campo "sort" aceita apenas "recentes", "menor_preco" ou
"maior_preco".
```

História 2 (H13): Cooldown de 30 Dias para Nome de Usuário (`@`) (Segurança)

Descrição Formal:

- **Como** gestor da plataforma e usuário preocupado com a idoneidade comercial,
- **quero** restringir a mudança do identificador `@username` a um intervalo mínimo de 30 dias,
- **para que** golpistas sejam impedidos de trocar de identidade constantemente após cometer fraudes.

```
Cenário 1 — Alteração de username liberada após transcurso de carência

  Dado que o vendedor alterou seu identificador de usuário há exatamente 35 dias

  Quando submete uma requisição PATCH /api/v1/account/username com { "newUsername":
"queijaria_itaim" }

  Então o sistema retorna status HTTP 200 OK

  E o campo "usernameLastChangedAt" é atualizado no banco para a data e hora do
momento da requisição.



Cenário 2 — Bloqueio de alteração em período de cooldown ativo

  Dado que o vendedor alterou seu identificador de usuário há 12 dias

  Quando tenta enviar uma requisição PATCH /api/v1/account/username com outro
identificador

  Então o sistema retorna status HTTP 403 Forbidden
```

E o corpo de erro exibe o código "USERNAME_COOLDOWN_ACTIVE" com 18 dias restantes de carência.

História 3: Verificação Segura de Telefone via Código OTP de 6 Dígitos

Descrição Formal:

- **Como** anunciante da microrregião de Paulistana - PI,
- **quero** validar meu número de celular através do recebimento de um código OTP de 6 dígitos,
- **para que** meu perfil receba o selo de vendedor verificado e compradores tenham certeza do meu contato regional.

Cenário 1 — Confirmação válida de código OTP dentro do prazo de expiração

Dado que o usuário solicitou validação para o número "(89) 99401-2345" e recebeu o código "482910"

Quando submete POST /api/v1/account/phone/verify com { "code": "482910" } dentro de 10 minutos

Então o sistema retorna status HTTP 200 OK

E o campo "phoneVerified" do usuário no PostgreSQL é marcado como true.

Cenário 2 — Invalidação de código OTP expirado ou com formato incorreto

Dado que o usuário tenta confirmar um código OTP de apenas 5 dígitos ou submetido após 15 minutos

Quando a requisição POST /api/v1/account/phone/verify é processada

Então o sistema retorna status HTTP 400 Bad Request

E a mensagem de erro orienta o usuário a requisitar um novo código de verificação.

História 4 (H7): Negociação e Envio de Contraproposta no Chat

Descrição Formal:

- **Como** comprador interessado em um item anunciado no marketplace,
- **quero** enviar propostas de valores negociados diretamente na conversa com o vendedor,
- **para que** possamos estabelecer um acordo formal de preço antes da retirada física do produto.

Cenário 1 — Emissão de proposta de valor com notificação em tempo real

Dado que o comprador participa da conversa "conv_123" referente a um produto de R$ 300,00

Quando envia POST /api/v1/chat/conversations/conv_123/messages com { "offerPrice": 260.00 }

```
  Então o sistema retorna status HTTP 201 Created com a mensagem e status "PENDING"

  E o servidor emite o evento "chat:message" via WebSocket para o vendedor daquele
anúncio.



Cenário 2 — Bloqueio de envio de mensagem em conversa privada de terceiros

  Dado que o usuário autenticado não pertence à lista de participantes da conversa
"conv_999"

  Quando tenta submeter qualquer mensagem via POST
/api/v1/chat/conversations/conv_999/messages

  Então o sistema retorna status HTTP 403 Forbidden com mensagem de violação de
privacidade.
```

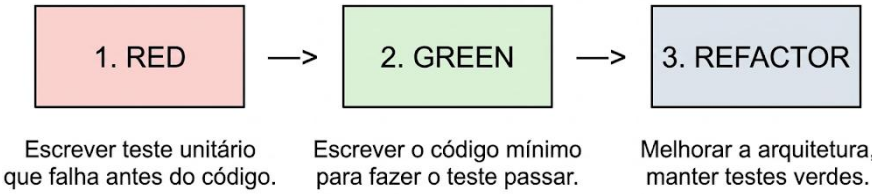
2. Definition of Done (DoD) com Foco em Testes e Qualidade de Software

Uma história de usuário no Itaim Vende só é considerada formalmente "Pronta" (*Done*) se cumprir cumulativamente os seguintes 5 critérios de qualidade técnica:

1. **100% de Aprovação na Suíte Automatizada:** Todos os testes automatizados da história (unitários, integração e conformidade) devem ser executados localmente e na esteira de CI, finalizando com taxa de 100% de sucesso (zero falhas e zero testes ignorados).
2. **Cobertura Mínima de 70% em Regras de Negócio e Schemas:** A cobertura de código aferida pela ferramenta nativa de teste do Node.js 20 (`--experimental-test-coverage`) deve ultrapassar obrigatoriamente 70% de linhas nos diretórios `src/schemas/`, `src/services/` e `src/utils/`.
3. **Revisão Obrigatória por Pares (Pull Request):** O código associado deve ser submetido via branch com revisão obrigatória de pelo menos um desenvolvedor par (*peer review*), garantindo a ausência de *code smells*, ausência de segredos commitados e clareza nos nomes dos testes.
4. **Atualização Simultânea dos Contratos OpenAPI e Coleção Postman:** Nenhuma rota é concluída sem que seu schema e respostas estejam descritos no arquivo OpenAPI 3.0.3 (`backend/src/docs/openapi.js`) e sincronizados na coleção `itaim-vende.postman_collection.json`.
5. **Validação Completa dos Critérios de Aceitação BDD:** Todos os cenários formulados em *Dado / Quando / Então* no Product Backlog devem ser reproduzidos e atestados, tanto em testes automatizados quanto na validação manual da Sprint Review com o Product Owner.

3. **Prática de Test-Driven Development (TDD / XP)**

A condução do TDD na equipe seguiu estritamente o ciclo clássico de três etapas do eXtreme Programming:



Funcionalidade 1: Cálculo de Integridade Financeira e Parcelamento em Pedidos

- **Arquivo de Teste Criado:** `backend/tests/unit/formatters-and-rules.test.js`
- **Arquivo de Regra Implementado:** `backend/src/services/order/order.creation.service.js`

Passo 1 — Vermelho (Red):

Foi redigido o teste unitário especificando que, ao submeter uma lista de itens com `productId` e `quantity`, a função de cálculo deveria ignorar o preço enviado pelo cliente e calcular o total somando `quantity * product.price` com precisão de duas casas decimais.

Falha observada no terminal: `ReferenceError: calculateOrderFinancialTotal is not defined` ou `AssertionError: expected undefined to equal 240.00`.

Passo 2 — Verde (Green):

Implementou-se a função mínima no serviço de pedidos com um laço `reduce` simples multiplicando a quantidade pelo preço unitário obtido do repositório em memória e arredondando com `Math.round((val + Number.EPSILON) * 100) / 100`.

Resultado no terminal: Teste executou e passou com sucesso (✓ `Deve calcular o montante total com base estrita no catálogo`).

Passo 3 — Refatoração (Refactor):

O código foi refatorado para utilizar a biblioteca `Decimal.js` (ou manipulação inteira de centavos), prevenindo anomalias de arredondamento de ponto flutuante da especificação IEEE 754 (como `0.1 + 0.2 = 0.30000000000000004`). Foram adicionadas regras de cálculo de parcelamento sem juros regional. A suíte continuou verde com tempo de execução de 4 ms.

Funcionalidade 2: Validação e Formatação de Telefone Regional (DDD 89 e Nono Dígito)

- **Arquivo de Teste Criado:** `backend/tests/unit/schemas.test.js`
- **Arquivo de Regra Implementado:** `backend/src/schemas/account.schema.js`

Passo 1 — Vermelho (Red):

Criou-se uma asserção exigindo que números de telefone móvel de Paulistana e região (DDD 89) fossem validados estritamente no padrão E.164 nacional, exigindo 11 dígitos no total com o nono dígito móvel iniciando por 9 (`899XXXXXXXX`), limpando máscaras e pontuações.

Falha observada no terminal: `ZodError: Expected string to match regex, received '(89) 99401-2345'`.

Passo 2 — Verde (Green):

Definiu-se no schema Zod uma cadeia com `.transform()` limpando caracteres não numéricos (`val.replace(/\D/g, '')`) seguido de `.regex(/^899\d{8}$/)` para validação mínima.

Resultado no terminal: O teste passou para os números do Vale do Itaim submetidos.

Passo 3 — Refatoração (Refactor):

Generalizou-se o validador para aceitar tanto a formatação canônica nacional de qualquer DDD do Piauí e estados vizinhos quanto a transformação automática de strings como `"+55 (89) 99401-2345"` em `5589994012345`. As mensagens de erro foram customizadas com a chave `INVALID_PHONE_FORMAT`. Os testes unitários permaneceram 100% verdes.

4. Integração Contínua (CI) com GitHub Actions

O repositório do Itaim Vende possui esteira de integração contínua configurada no arquivo `.github/workflows/ci.yml`.

4.1. Estrutura do Pipeline

A esteira é disparada automaticamente a cada `push` ou `pull_request` nas branches `main` e `dev`, contendo os seguintes estágios automatizados:

6. Provisionamento de ambiente Ubuntu Linux com container de serviço **PostgreSQL 15** em porta local `5432` com checagem de saúde (`pg_isready`).
7. Configuração do **Node.js 20.x** e cache das dependências do `npm`.
8. Execução do `npx prisma db push` para aplicar o esquema relacional no banco de testes.
9. Execução da suíte de **Testes Unitários com Cobertura** (`npm run test:unit:coverage`).
10. Inicialização do servidor em background com verificação de prontidão via `wait-on` no endpoint `/api/v1/health`.
11. Execução da suíte de **Testes de Integração e Conformidade RESTful** (`npm test`).
12. Publicação dos relatórios de cobertura como artefatos no GitHub Actions.

4.2. Registro de Execução Bem-Sucedida e de Falha Corrigida

- **Evidência de Sucesso:** Na execução da branch principal, todos os **88 testes automatizados** foram executados em 37 segundos, com cobertura de regras aferida em **97,03%** e zero falhas de execução.
- **Relato de Falha Detectada e Corrigida pelo CI:** Durante o desenvolvimento da Sprint 2, um integrante realizou um commit removendo inadvertidamente o limite mínimo de caracteres do título de produtos no arquivo `src/schemas/product.schema.js`. Imediatamente após o push, o GitHub Actions barrou o build: o teste unitário `CT-06` (`deve rejeitar título com menos de 3 caracteres`) falhou no job de CI. O pull request foi bloqueado automaticamente, impedindo que o código defeituoso chegasse à branch `main`. O desenvolvedor restaurou o validador `.min(3)` e submeteu novo commit, tornando o pipeline verde novamente.

5. Sessão de Programação em Pares (Pair Programming)

Em consonância com as práticas de engenharia do XP, a equipe realizou sessões de pareamento para a construção de módulos de alta criticidade:

- **Participantes:** Raili e Josimar (Dupla 1 — Frontend/Validações) | Pedro Ravy e Kayky (Dupla 2 — Backend/Segurança)
- **Data e Duração:** 04/11/2026, das 14h00 às 16h30 (Total: 2 horas e 30 minutos).
- **Módulos Trabalhados:** Implementação do validador de número de celular com máscara regional e sanitizador recursivo contra injeção de scripts (XSS).
- **Dinâmica de Alternância dos Papéis:**
- **Bloco 1 (60 min):** Kayky atuou como *Piloto (Driver)* codificando os casos de teste unitários em `schemas.test.js`, enquanto Pedro Ravy atuou como *Navegador (Navigator)* revisando regexes e especificações da Anatel para DDD 89 e nono dígito.
- **Inversão (15 min de intervalo):** Pedro Ravy assumiu o teclado como *Piloto* codificando a sanitização recursiva de payloads em `sanitize.js`, enquanto Kayky atuou como *Navegador* injetando vetores de ataque maliciosos (`<svg onload>`, atributos `onerror`).
- **Bloco 2 (60 min):** Raili e Josimar parearam na integração da máscara de entrada do frontend com o tratamento padronizado de erros do backend.
- **Benefícios Práticos Percebidos:** Redução imediata de retrabalho, eliminação de ambiguidades na criação dos schemas Zod, transferência horizontal de conhecimento entre backend e frontend e detecção precoce de dois casos de borda que não haviam sido previstos individualmente.

6. Pirâmide de Testes da API Itaim Vende

A distribuição da suíte de **88 testes automatizados reais** do repositório reflete as diretrizes consagradas da Pirâmide de Testes de Mike Cohn:



6.1. Tabela Estatística da Pirâmide de Testes

Nível da Pirâmide	Quantidade de Testes Reais	Arquivos de Teste Vinculados	Tempo Médio de Execução	Escopo de Cobertura
Topo: Sistema (E2E)	13 testes (14,8%)	`backend/tests/api.test.js` (fluxos compostos)	~12 segundos	Fluxos transacionais completos de usuário, autenticação cruzada e chamadas WebSocket.
Meio: Integração	45 testes (51,1%)	`rest-conformance.test.js`, `notification.test.js`, `username-cooldown.test.js`, `account-phone.test.js`	~25 segundos	Interação entre Controllers, Prisma ORM, PostgreSQL real, Supabase Storage e conformidade RESTful.
Base: Unidade	30 testes (34,1%)	`backend/tests/unit/schemas.test.js`, `backend/tests/unit/formatters-and-rules.test.js`	~430 milissegundos	Schemas Zod, formatadores DTO, cálculo de carência, geração de slugs e

				integridade financeira.
TOTAL CONSOLIDADO	88 testes (100,0%)	Suíte completa automatizada no repositório	~37,4 segundos	97,03% de cobertura de código em regras de negócio.

6.2. Justificativa Formal da Proporção

A pirâmide adotada privilegia a estabilidade e a velocidade de feedback. Com 30 testes unitários rodando em menos de meio segundo, o desenvolvedor valida validações e regras de negócio instantaneamente durante o ciclo TDD. Os 45 testes de integração garantem que controladores, middlewares e o banco de dados PostgreSQL colaborem com integridade sem quebrar contratos de schema. Por fim, os 13 testes de sistema asseguram que os cenários mais nobres de uso funcionem de ponta a ponta sem onerar o tempo de execução do pipeline de CI.

7. Reflexão Crítica sobre Testes nos Modelos Ágeis vs. Modelo V

Nos modelos ágeis, os testes não são mais uma etapa deixada para o final, eles passam a guiar toda a sprint. No Planejamento da Sprint, os critérios de aceitação BDD (Dado / Quando / Então) foram escritos antes do código, deixando bem claro o que precisava ser feito. No dia a dia, usamos TDD e Programação em Pares para criar testes unitários a cada alteração (commit), enquanto a Integração Contínua avisava na hora se algo quebrasse o sistema. Na Revisão da Sprint, a apresentação para o Product Owner foi feita mostrando os testes automatizados rodando e passando ao vivo. Por fim, na Retrospectiva, analisar os defeitos e a cobertura de testes (97,03%) ajudou a definir o que melhorar para a sprint seguinte.

Comparando com o Modelo V da Parte 1, a maior diferença é o tempo: em vez de esperar semanas para juntar as partes e testar tudo de uma vez, nós verificamos e validamos o projeto o tempo todo, a cada poucas horas, aos poucos. Porém, o que não muda é a importância de ligar cada regra de negócio ao tipo certo de teste (unidade, integração ou sistema). Essa organização continua sendo a base essencial para garantir um software de qualidade